

Interview Planner Project Guide

Integrated 4-phase learning document for understanding the architecture, backend, frontend, and full user journey of your project.

Phase 1: High-Level Architecture

This project is a full-stack AI interview preparation application with a clear separation between frontend, backend, database, and AI service integration.

Project Split

- **Frontend:** React + Vite application inside **Frontend/**
- **Backend:** Node.js + Express API inside **Backend/**
- **Database:** MongoDB using Mongoose
- **AI:** Gemini API through **@google/genai**

```
React frontend <-> Express backend <-> MongoDB
                        |
                        -> Gemini API
```

Frontend Entry

- **Frontend/src/main.jsx** boots React
- **Frontend/src/App.jsx** wraps the app with **AuthProvider**, **InterviewProvider**, and the router

Backend Entry

- **Backend/server.js** loads environment variables, connects to MongoDB, and starts the server on port **3000**
- **Backend/src/app.js** configures Express, middleware, and API routes

How Frontend And Backend Communicate

- The frontend uses **Axios** with base URL **http://localhost:3000**
- Authentication and feature calls are organized into **auth.api.js** and **interview.api.js**
- Requests use **withCredentials: true** so JWT cookies are sent automatically

How Gemini Fits In

- The frontend never talks directly to Gemini
- The backend sends structured prompt data to Gemini from **Backend/src/services/ai.service.js**
- Gemini returns JSON containing match score, interview questions, skill gaps, roadmap, and title

Phase 1 takeaway: this is a real full-stack architecture with client state, protected API routes, MongoDB persistence, and backend-controlled AI integration.

Phase 2: Backend Mechanics And API Routes

The backend follows a clean route -> middleware -> controller -> service -> model structure.

Boot Flow

- **server.js** starts the application
- **database.js** connects to MongoDB using **MONGO_URI**
- **app.js** configures JSON parsing, cookies, CORS, and error handling

Auth Routes

- **POST /api/auth/register**
- **POST /api/auth/login**
- **GET /api/auth/logout**
- **GET /api/auth/get-me**

Interview Routes

- **POST /api/interview/** generates a report
- **GET /api/interview/** gets report history
- **GET /api/interview/report/:interviewId** gets one report
- **POST /api/interview/resume/pdf/:interviewReportId** generates a resume PDF

Middleware

- **auth.middleware.js** reads the JWT from cookies, checks blacklist, verifies it, and attaches **req.user**
- **file.middleware.js** uses Multer memory storage, validates file type, and limits uploads to **5MB**

Controllers

- **auth.controller.js** handles registration, login, logout, and current-user retrieval
- **interview.controller.js** validates inputs, parses resumes, calls Gemini, stores reports, fetches reports, and creates downloadable PDFs

Service Layer

- **ai.service.js** contains the AI logic instead of mixing it into controllers
- It uses **Zod + zod-to-json-schema** to force structured Gemini responses
- It also generates HTML resumes and converts them to PDF with **Puppeteer**

Persistence

- **interviewReport.model.js** stores inputs, generated outputs, user ownership, and timestamps
- This makes interview reports persistent application data instead of temporary AI responses

```
POST /api/interview/  
-> auth middleware  
-> upload middleware  
-> controller validation  
-> resume parsing  
-> Gemini service  
-> MongoDB save  
-> response to frontend
```

Phase 2 takeaway: your backend is organized like a real production service with layered responsibilities and secure AI access from the server side.

Phase 3: Frontend Layout, Routing, And State Management

The frontend uses React, React Router, Context, and custom hooks to manage auth, interview state, and page-level workflows.

App Composition

- **main.jsx** renders the app
- **App.jsx** wraps the app in **AuthProvider**, **InterviewProvider**, and **RouterProvider**

Routing

- **/login** and **/register** are public
- **/** and **/interview/:interviewId** are protected
- **Protected.jsx** blocks unauthenticated access and redirects users to login

Auth State

- **auth.context.jsx** stores **user** and **loading**
- **useAuth.js** contains **handleLogin**, **handleRegister**, **handleLogout**, and initial **getMe()**
- **getMe()** restores the session after refresh using the cookie already stored by the backend

Interview State

- **interview.context.jsx** stores **report**, **reports**, and **loading**
- **useInterview.js** contains report generation, fetching, listing, and PDF download logic

Frontend API Layer

- **auth.api.js** handles auth HTTP calls
- **interview.api.js** handles feature HTTP calls
- Pages stay cleaner because API requests are not embedded directly into UI components

Main Pages

- **Login.jsx** and **Register.jsx** collect credentials and delegate logic to hooks
- **Home.jsx** is the main orchestration page where the user submits the job description and profile data
- **Interview.jsx** is the report detail page that reads the route param and displays the stored report

Frontend Data Flow

```
Page component
-> custom hook
-> context state update
```

- > API service
- > backend response
- > UI re-render

Phase 3 takeaway: the frontend is organized by pages, hooks, context, and service files, which keeps state and UI responsibilities cleanly separated.

Phase 4: Full User Journey End-To-End

This phase connects the whole system into one workflow from login to report generation and resume PDF download.

Step 1: App Boot And Session Restore

- The app starts and `useAuth()` calls `getMe()`
- If the JWT cookie is valid, backend returns user data and the frontend restores the session

Step 2: Protected Access

- Protected pages are available only if `user` exists in auth context
- Otherwise the user is redirected to login

Step 3: User Input

- On `Home.jsx`, the user enters a `jobDescription`
- The user either uploads a resume or types a self-description
- The frontend validates required input, file type, and file size before sending the request

Step 4: Backend Processing

- The request goes through auth middleware and upload middleware
- The controller extracts and validates the request body
- If a resume exists, it is parsed into text before going to Gemini

Step 5: AI Generation

- Gemini receives the resume text, self-description, and job description
- It returns a structured interview report with score, questions, gaps, plan, and title

Step 6: Save And Display

- The backend stores the report in MongoDB using the current user ID
- The frontend stores the response and navigates to the report detail page
- `Interview.jsx` fetches the report again by route parameter and displays it

Step 7: Report History And Resume PDF

- The home page can show recent reports from the database
- The report page lets the user download a resume PDF generated from saved report data
- The backend uses Gemini to create resume HTML and Puppeteer to export it as PDF

```
User logs in
-> session restored with getMe()
-> enters job description + resume/self-description
```

```
-> backend authenticates and parses input
-> Gemini generates structured report
-> MongoDB stores report
-> frontend shows report
-> user downloads tailored resume PDF
```

Phase 4 takeaway: your project behaves like a complete product, not a simple demo, because it includes authentication, validation, AI orchestration, persistence, history, and downloadable output.

Final Summary

Your Interview Planner project demonstrates a practical full-stack architecture with:

- React + Vite for UI and routing
- Context + hooks for state management
- Express for backend APIs
- JWT cookie-based authentication
- MongoDB for persistent report storage
- Gemini for structured AI generation
- Puppeteer for resume PDF export

If you can explain these four phases clearly, you can confidently walk an interviewer through the system architecture and the end-to-end user journey of this project.

Generated from the local Interview Planner project walkthrough.